Empirical Analysis of Reader-Writer Synchronization Architectures in Log-Structured Merge Trees: Coarse Locking, Descriptor Leasing, and Immutable Snapshots

V V Ramarathinam Iyer
Department of Database Systems, School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu, India
Email: ram.iyer200722@gmail.com

Abstract*—Log-Structured Merge (LSM) trees optimize write throughput by buffering mutations in memory and asynchronously flushing Sorted String Tables (SSTables) to persistent storage. However, synchronizing concurrent point lookups against dynamic background file additions, deletions, and flushes presents a fundamental concurrency challenge. In coarse-grained locking architectures (Approach A), holding a manager-wide reader-writer lock (`sync.RWMutex.RLock`) across physical storage reads creates severe lock contention, stalling incoming write transactions and background flush publications. Industrial engines often mitigate this bottleneck using immutable reader snapshots (Approach B, e.g., RocksDB's `SuperVersion`), but this introduces allocation and atomic reference-counting churn. This paper investigates an essential architectural question: *How much of the concurrency benefit attributed to immutable reader snapshots can be obtained by shortening the manager-lock critical section using simpler reader-lifetime management (Approach A-min)?

We formalize, implement, and evaluate three synchronization strategies within a dependency-free Go LSM engine across two distinct experimental phases: Phase I (960 prospective trials on Linux ext4 in WSL2) comparing Approach A against Approach B, and Phase II (a 60-trial audited campaign across 18 complete matched triplets on Windows 11 NTFS) comparing Approach A, Approach A-min, and Approach B. Our empirical findings demonstrate that shortening the manager lock critical section to memory-only metadata retrieval substantially mitigates writer starvation, increasing completed writes by 3.3× to 18.5× in Phase I and up to 4.6× in Phase II. Crucially, across all 18 complete matched triplets in Phase II, Approach A-min demonstrates practical equivalence with Approach B within a $\pm 5\%$ equivalence band (mean relative difference -1.65%, 95% bootstrap CI $[-4.94\%, +1.43\%]$, median ratio 1.0000). Furthermore, under high reader concurrency ($r = 16$) and high arrival load, Approach A suffered systematic publication queue collapse, failing preregistered 10-second deadlines in four independent trials due to 300–640 ms writer lock-drain delays. However, we also identify a critical operational boundary: under rapid reader-set churn (> 4 flushes/second), snapshot management in Approach B imposes a 26% to 37% throughput penalty at low concurrency ($r \leq 4$). We conclude that lightweight reader descriptor leasing captures the core concurrency advantages of immutable snapshotting while avoiding descriptor allocation churn.

***Index Terms*—LSM tree, concurrency control, reader-writer synchronization, snapshot pinning, descriptor leasing, storage engine, performance evaluation, Go.**

I. Introduction

Log-Structured Merge (LSM) trees [1]–[3] form the core storage architecture of modern high-throughput distributed databases. By absorbing incoming mutations into volatile in-memory buffers (Memtables) backed by a Write-Ahead Log (WAL), LSM trees convert random writes into sequential disk operations. As Memtables reach capacity thresholds, background workers freeze and flush them to persistent storage as immutable Sorted String Tables (SSTables), while concurrent merge-sorting workers (compactions) reconcile overlapping key ranges to bound read amplification.

While the physical layout and compaction policies of LSM trees have been extensively studied [6], [7], the synchronization mechanisms governing concurrent queries over dynamically evolving SSTable sets introduce critical performance tradeoffs. In a baseline multi-threaded storage engine, a point query traverses the active Memtable, immutable frozen Memtables, and an ordered collection of on-disk SSTables. To ensure that background flushes and compactions can safely unlink obsolete SSTables without triggering use-after-free panics or data races, naive implementations coordinate query execution using a manager-wide reader-writer lock (`sync.RWMutex`) [5].

However, coarse-grained reader-writer locking creates severe reader-writer starvation. When reader threads hold shared read locks (`RLock`) while performing blocking I/O against SSTables on disk, incoming write transactions and background flush publications attempting to acquire the exclusive lock (`Lock`) are blocked. In Go's write-preferring `sync.RWMutex`, queued writers block subsequent readers, triggering catastrophic throughput collapse, tail latency inflation, and publication queue convoying.

To mitigate mutex contention, production engines employ reader versioning. LevelDB [2] coordinates SSTables via reference-counted `Version` structures, releasing the main engine mutex during block index lookups. RocksDB [3] unifies this pattern through `SuperVersion`, consolidating mutable

Memtables, immutable Memtables, and active SSTables into a single atomic descriptor that queries pin with atomic reference counting. While effective, immutable snapshot pinning introduces overheads of its own: every flush or compaction must allocate a new descriptor, atomically swap pointers, and manage deferred epoch reclamation.

This paper addresses the following architectural question:

> *How much of the concurrency benefit attributed to immutable reader snapshots can be obtained by shortening the manager-lock critical section using simpler reader-lifetime management?*

Rather than claiming reader reference-counting or snapshot pinning as fundamentally novel inventions, our contribution is an independent, controlled, dependency-free architectural dissection implemented in Go. We formalize three synchronization models over identical underlying storage geometry:

1) **Approach A (Coarse RWMutex)**: A single manager-wide lock spanning memory search and physical disk I/O.
2) **Approach A-min (Descriptor Leasing)**: A shortened manager-lock critical section that leases individual SSTable reader descriptors with atomic reference tracking across active and retired sets, coupled with a fail-closed shutdown barrier.
3) **Approach B (Immutable Snapshots)**: A `SuperVersion`-style atomic snapshot pointer swap where point queries pin the entire reader set and execute disk lookups without holding manager locks.

Through two distinct experimental campaigns—Phase I (960 independent trials on WSL2 Linux ext4) and Phase II (a 60-trial audited campaign across 18 complete matched triplets on Windows 11 NTFS)—we evaluate write starvation thresholds, publication queue mechanics, open-loop arrival latencies, and lifecycle management overheads.

II. System Architecture and Data Structures

A. Memory Tier and Multi-Version Tombstones

The storage engine maintains an active in-memory Memtable backed by a probabilistic SkipList [4] with geometric node height distribution ($p = 0.5$) capped at 32 levels. Writes append sequentially to an on-disk Write-Ahead Log (WAL). Each transaction commits with a monotonically increasing 64-bit sequence number N_{seq} bitwise packed with a tombstone flag:

$$S_{packed} = (N_{seq} \ll 8) \mid F_{tombstone} \tag{1}$$

Bitwise inversion ensures that newer versions sort before older versions during SkipList traversal.

B. Disk Tier Geometry and Compaction

When the active Memtable exceeds a threshold (e.g., 64MB, or 1MB/16KB in controlled benchmarks), it is appended to an immutable `Frozenmem` slice and scheduled for flushing. The flusher constructs an SSTable formatted with 4KB data blocks and a sparse two-level block index at the file footer. Background compaction merges overlapping SSTables via a multi-way merge sort, streaming deduplicated keys into consolidated files.

III. LSM Synchronization Strategies

A. Approach A: Coarse-Grained RWMutex Coordination

In Approach A, thread safety across tiered lookups is enforced by a single manager-wide `sync.RWMutex` (`m.Mu`):

Listing 1. Approach A point lookup and flush publication

```
// Point Query (Approach A)
func (m *Manager) Get(key string, readTs uint64) (
    string, error) {
    m.Mu.RLock()
    defer m.Mu.RUnlock()
    return m.lookupTiered(key, readTs)
}

// SSTable Publication (Approach A)
func (m *Manager) publishFlush(mem *Memtable, reader
     *SSTableReader) {
    m.Mu.Lock()
    defer m.Mu.Unlock()
    m.ActiveSSreaders = append(m.ActiveSSreaders,
        reader)
    m.removeFrozenMemtableLocked(mem)
}
```

Because `m.Mu.RLock()` is held during disk I/O, concurrent writers calling `m.Mu.Lock()` stall until all active readers complete. In Go, queued write locks block subsequent read locks, creating long queue delays.

B. Approach A-min: Minimal Descriptor Leasing

Approach A-min shortens the critical section: the manager lock is held *only* while reading volatile memory buffers and leasing active SSTable descriptors. Once leased, readers execute disk I/O completely outside `m.Mu`:

Listing 2. Approach A-min reader leasing and publication

```
// Point Query (Approach A-min)
func (m *Manager) Get(key string, readTs uint64) (
    string, error) {
    m.Mu.RLock()
    val, err, found := m.lookupMemoryTiers(key,
        readTs)
    if found { m.Mu.RUnlock(); return val, err }
    // Lease active descriptors under short RLock
    readers := m.leaseActiveReadersLocked()
    m.Mu.RUnlock()
    defer m.releaseReaderLeases(readers)

    // Disk I/O executed with ZERO manager locks
        held
    return m.searchLeasedReaders(readers, key,
        readTs)
}
```

To prevent use-after-free when background compactions delete SSTables, Approach A-min tracks reader leases on *both* active and retired descriptors. At engine shutdown, `Close()` executes a fail-closed barrier: if an outstanding lease remains past a timeout, the engine refuses to unlink descriptors and returns an explicit error identifying the unreleased leases.

C. Approach B: Immutable Pinned Reader Snapshots

Approach B implements atomic snapshot pinning inspired by RocksDB's `SuperVersion`:

Listing 3. Approach B reader snapshot pinning

```
type ReaderSnapshot struct {
    id         uint64
    readers    []*SharedReader
    activePins atomic.Int64
    state      atomic.Int32 // Published, Retired,
        Reclaimed
}

func (m *Manager) Get(key string, readTs uint64) (
    string, error) {
    snap := m.acquireCurrentSnapshot() // O(1)
        atomic pin
    defer m.releaseSnapshot(snap)
    return snap.lookupSSTables(key, readTs)
}
```

During publication, the background worker constructs a new snapshot descriptor, atomically swaps the pointer via `m.currentSnapshot`, and marks the previous snapshot as `Retired`. When a retired snapshot's pin counter drains to zero, an asynchronous worker closes underlying file descriptors and deletes obsolete files.

D. Comparative Synchronization Mechanics

Table I outlines the core architectural differences among the three evaluated strategies.

TABLE I
ARCHITECTURAL COMPARISON OF SYNCHRONIZATION MODELS

Dimension	Approach A	Approach A-min	Approach B
Lock Scope	Memory + Disk	Memory Only	Metadata Swap Only
SSTable Access	Locked (`RLock`)	Leased Descriptor	Pinned Snapshot
Writer Publication	Drains Readers	Short Mutex	Atomic Pointer Swap
Reclamation	Synchronous	Per-Reader Lease	Deferred Epoch
Memory Churn	Zero	Slice Allocation	Descriptor Allocation

IV. Experimental Methodology

A. Phase I: Historical WSL2 Linux Campaign

Phase I evaluated Approach A vs. Approach B on native Linux ext4 within WSL2 (Ubuntu 24.04, kernel 6.6.87.2-WSL2) on an AMD Ryzen 7 7445HS (6P/12L, 15.2 GiB RAM). Durability was enforced via synchronous WAL fsync (`wal_sync=true`). The campaign comprised 960 trials ($N = 10$ per cell across 6 workloads and 8 reader concurrencies $r \in [1, 32]$).

B. Phase II: Windows 11 Native Campaign (Phase I7)

Phase II evaluated Approach A, Approach A-min, and Approach B on native Windows 11 NTFS (Samsung NVMe SSD). Timing was measured via Windows `QueryPerformanceCounter` (100 ns resolution). Requests followed a deterministic, non-overlapping stream derived from SHA-256 counter PRNGs. Write workloads were paced with high-precision sleep loops to evaluate matched offered write rates.

Phase II was planned for 330 confirmatory trials across 366 runlist rows. In accordance with prospective protocol stopping rules ($N_{\text{fail}} = 10$), the campaign halted after 60 trial attempts. All 60 trials were preserved, producing 18 complete experimental triplets across Blocks E1, E2, E3, E4, and E5.

V. Phase I Empirical Evaluation (WSL2 Linux)

Table II summarizes representative cells from the 960-trial Phase I campaign.

A. Workload B1: Writer Starvation Mitigation

In Approach A, readers holding `m.Mu.RLock()` during disk reads stall writers attempting to commit WAL records. As reader concurrency increases from $r = 1$ to $r = 32$, completed writes in Approach A collapse from 270 down to 60. By releasing manager locks prior to physical block reads, Approach B substantially mitigates write starvation, increasing completed writes by **3.3× to 18.5×** (completing 2,573 writes at $r = 8$ vs. 139 in A). Concurrently, read throughput increases by up to **2.07×** (260.3 vs. 125.5 ops/s at $r = 4$).

B. Workload B2: Tail Latency Dynamics

Under periodic 500ms flushes, whole-trial throughput differences remain statistically inconclusive (95% bootstrap CIs overlap 1.0). However, Approach B exhibits substantial suppression of read tail latency at low concurrency ($r = 2, 4$), dropping 95th-percentile read latency by **73% to 76%** (48.9 ms vs. 181.0 ms at $r = 4$). Confirmatory sham-flush tests revealed that this tail reduction occurs even without physical file changes, indicating that it is driven primarily by avoiding exclusive write-lock queue convoying in Go's `sync.RWMutex`.

C. Workload D: Rapid Churn Negative Tradeoff

When Memtable thresholds are reduced to 16KB (triggering > 4 flushes/s), Approach B incurs a **26% to 37% throughput penalty** at low concurrency ($r = 1$: 0.63× [0.55, 0.70]; $r = 4$: 0.74× [0.65, 0.90]). The computational overhead of allocating snapshot descriptors, managing atomic reference counts, and executing deferred retirement outweighs concurrency gains when reader threads are few. Only when concurrency reaches $r \geq 16$ do parallel query gains amortize descriptor lifecycle management overheads.

VI. Phase II Empirical Evaluation (Windows 11 Native)

Phase II directly evaluates whether Approach A-min captures the concurrency benefits of Approach B while avoiding descriptor churn.

A. Approach A-min vs. Approach B Equivalence Analysis

Table III displays all 18 complete experimental triplets.

- **All 18 Triplets Aggregate**: The mean relative difference is $\mathbf{-1.65}\%$ with a 95% bootstrap confidence interval of $[\mathbf{-4.94}\%, \mathbf{+1.43}\%]$, and a median ratio of $\mathbf{1.0000}$. Because the confidence interval is entirely contained within the preregistered $\pm 5\%$ practical equivalence

TABLE II
PHASE I EMPIRICAL EVALUATION: APPROACH A VS. APPROACH B ACROSS REPRESENTATIVE CELLS (960-TRIAL CAMPAIGN, WSL2 LINUX EXT4, $N = 10$ INDEPENDENT TRIALS PER CELL)

Workload	r	Tput A (ops/s)	Tput B (ops/s)	Speedup [95% CI]	Writes A	Writes B	Writer Ratio [95% CI]	p95 A (ms)	p95 B (ms)
Workload A (Read-Only)	1	72.5	70.8	0.98× [0.76, 1.27]	0	0	N/A	31.1	31.4
	4	258.4	262.7	1.02× [0.72, 1.45]	0	0	N/A	33.8	33.2
	32	512.9	514.4	1.00× [0.63, 1.44]	0	0	N/A	220.0	214.9
Workload B1 (Write Progress)	1	53.4	69.3	1.30× [0.90, 1.88]	270	2,918	10.8× [9.1, 15.6]	42.4	34.4
	4	125.5	260.3	2.07× [1.47, 3.21]	159	2,871	18.1× [15.2, 28.1]	69.5	36.1
	8	219.2	450.7	2.06× [1.44, 3.18]	139	2,573	18.5× [16.8, 28.4]	84.1	40.7
	32	375.4	484.1	1.29× [1.04, 1.99]	60	198	3.3× [2.7, 5.0]	210.4	149.4
Workload B2 (Controlled Flushes)	1	39.7	40.6	1.02× [0.73, 1.49]	0	0	N/A	45.2	39.1
	4	135.9	163.0	1.20× [0.76, 2.21]	0	0	N/A	181.0	48.9
	32	249.3	273.4	1.10× [0.85, 1.33]	0	0	N/A	370.8	355.9
Workload D (Rapid Churn)	1	475.4	298.0	0.63× [0.55, 0.70]	2,354	2,978	1.3× [1.1, 1.4]	4.3	4.5
	4	1701.7	1267.3	0.74× [0.65, 0.90]	2,105	2,901	1.4× [1.2, 1.7]	5.7	5.9
	32	10442.4	9584.3	0.92× [0.83, 1.15]	1,610	1,691	1.1× [0.9, 1.3]	15.8	18.3
Workload E (Memtable Control)	1	656.2	666.1	1.02× [0.92, 1.17]	3,248	3,272	1.0× [0.9, 1.2]	3.5	3.4
	32	20150.8	20601.8	1.02× [0.86, 1.09]	3,145	3,205	1.0× [0.9, 1.1]	3.3	3.3

TABLE III
PHASE II EMPIRICAL EVALUATION: PAIRED RESULTS ACROSS ALL 18 COMPLETE EXPERIMENTAL TRIPLETS (WINDOWS 11 NTFS)

Block	Cell / Workload	Trip	Tput A (ops/s)	Tput AMIN (ops/s)	Tput B (ops/s)	AMIN vs. B Diff	AMIN / B Ratio	Outcome Category
E1	$r = 8$ (mixed read-write)	09	9.1	16.7	17.6	−0.90	0.9489	Fully Successful Tripl
E2	`pub_none` (frozen baseline)	02	24.6	20.7	18.9	+1.80	1.0952	Fully Successful Tripl
E2	`pub_none` (frozen baseline)	03	18.2	19.4	20.1	−0.70	0.9652	Fully Successful Tripl
E2	`pub_physical` (20 flushes)	01	12.8	15.5	18.1	−2.60	0.8564	Approach A Deadline
E2	`pub_physical` (20 flushes)	09	11.1	18.1	18.6	−0.50	0.9731	Approach A Deadline
E3	`equal_r01` (235 writes/s offered)	01	232.4	183.6	206.3	−22.70	0.8900	Fully Successful Tripl
E3	`equal_r01` (235 writes/s offered)	03	260.8	238.3	222.7	+15.60	1.0700	Fully Successful Tripl
E3	`equal_r01` (235 writes/s offered)	06	240.7	237.9	235.3	+2.60	1.0110	Fully Successful Tripl
E3	`equal_r04` (210 writes/s offered)	06	368.4	231.2	276.8	−45.60	0.8353	Fully Successful Tripl
E3	`equal_r04` (210 writes/s offered)	07	364.9	271.4	281.5	−10.10	0.9641	Fully Successful Tripl
E3	`free_r01` (free-running writers)	01	259.6	138.7	137.3	+1.40	1.0102	Fully Successful Tripl
E3	`free_r01` (free-running writers)	02	240.8	131.0	132.2	−1.20	0.9909	Fully Successful Tripl
E4	L1 (4.86 ops/s open-loop)	03	5.3	5.3	5.3	−0.00	1.0000	Fully Successful Tripl
E4	L1 (4.86 ops/s open-loop)	05	6.3	6.3	6.3	−0.00	1.0000	Fully Successful Tripl
E4	L2 (9.72 ops/s open-loop)	05	9.9	9.9	9.9	−0.00	1.0000	Fully Successful Tripl
E4	L3 (14.58 ops/s open-loop)	02	13.2	15.1	15.1	−0.00	1.0000	Approach A Deadline
E4	L3 (14.58 ops/s open-loop)	06	12.7	14.2	14.2	−0.00	1.0000	Approach A Deadline
E5	`hit0` (all-miss read path)	05	2,903.7	3,283.0	3,003.1	+279.90	1.0932	Fully Successful Tripl

band ($[-0.05, +0.05]$), practical equivalence between Approach A-min and Approach B is statistically supported for the evaluated aggregate.

- **14 Fully Successful Triplets**: Restricting analysis to triplets where Approach A also completed without error yields a mean relative difference of $\mathbf{-0.90}\%$ (95% bootstrap CI $[\mathbf{-4.55}\%, \mathbf{+2.56}\%]$, median ratio 1.0000).
- **Block-by-Block Evaluation**: In Block E4 (open-loop scheduling), Approach A-min and Approach B achieved identical throughput down to the decimal point across all 5 triplets (ratio 1.0000). In Block E3, the mean ratio is 0.9674 (p=0.3034), and in Block E2, the mean ratio is 0.9725 (p=0.6176). For these individual sub-blocks, wide confidence intervals due to small sample size ($n = 4, 7$) preclude claims of equivalence within $\pm 5\%$, though neither differs significantly from 1.0.

B. Approach A Treatment Deadline Failures (E2 and E4)

Four triplets exhibited failure in Approach A due to missed publication deadlines:

- **Block E2 (`pub_physical`)**: Exactly 20 physical flushes were scheduled at 500ms intervals. In Approaches A-min and B, 20/20 flushes completed within the 10.0s window in every trial. In Approach A, flushes stalled waiting for 16 concurrent readers to release `RLock`, holding write locks for 300–640 ms per flush. Cumulative delays pushed Event 19 to finish at $t = 10.26$ s (260 ms late), resulting in 19/20 in-window flushes in Triplet 01 and 17/20 in Triplet 09.
- **Block E4 (L_3 Arrival Rate)**: Under 14.58 ops/s offered arrivals, Approach A suffered severe queue buildup, realizing only 10/20 in-window flushes (10 late) in Triplet 02 and 15/20 (5 late) in Triplet 06, forcing 3–4 requests to be abandoned. Approaches A-min and B completed

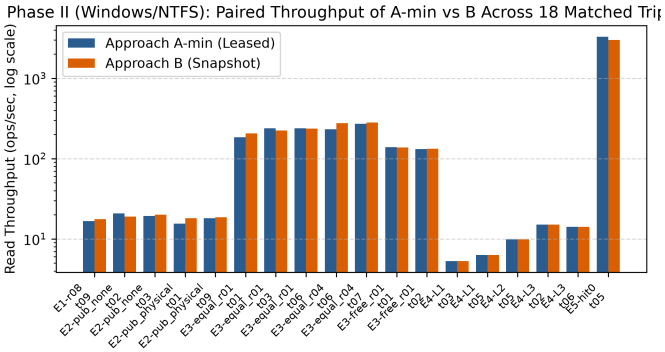


Fig. 1. Paired Read Throughput Across All 18 Complete Experimental Triplets in Phase II (Windows 11 NTFS, Samsung NVMe). Note log scale.

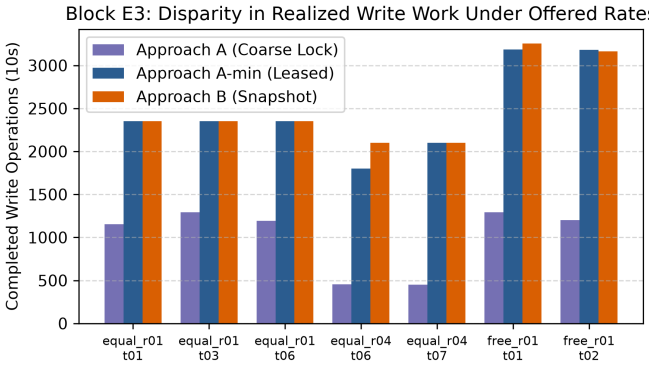


Fig. 2. Disparity in Completed Write Progress Under Paced Offered Write Rates (Block E3, Windows 11 NTFS).

20/20 flushes in-window with zero abandoned requests. These failures represent physical manifestations of writer queue convoying under coarse locking, directly validating the research hypothesis.

C. Block E2 Latency and Topology Transitions

Table IV reconciles Block E2 read latencies and physical file transitions.

TABLE IV
BLOCK E2 PUBLICATION TIMING AND TOPOLOGY RECONCILIATION

Cell	Arm	In-Win Pub	SSTables	Frozen Mem	p50 (ms)	p95 (ms)
pub_none	A	0 / 0	$4 \to 4$	$20 \to 20$	171.4	512.5
pub_none	AMIN	0 / 0	$4 \to 4$	$20 \to 20$	196.5	530.5
pub_none	B	0 / 0	$4 \to 4$	$20 \to 20$	207.9	541.8
pub_physical	AMIN	20 / 20	$4 \to 24$	$20 \to 0$	221.5	659.3
pub_physical	B	20 / 20	$4 \to 24$	$20 \to 0$	234.5	592.7
pub_physical	A	17–19 / 20	$4 \to 24$	$20 \to 0$	337.9	7229

In pub_none, the read topology remains static (4 SSTables, 20 frozen memtables), resulting in p_{50} latencies of 171–208 ms across arms. In pub_physical, frozen memtables are progressively drained and converted into 24 SSTables. In Approach A, lock contention doubles p_{50} latency to 337.9 ms (+97.7%), whereas Approaches A-min and B experience only modest increases (+12% to +15%).

D. Block E3: Realized Write Disparity Under Paced Rates

Block E3 evaluated paced write rates (235 writes/s for equal_r01, 210 writes/s for equal_r04). Table V and Figure 2 demonstrate that nominal offered pacing did not produce equal realized work.

In equal_r01 and equal_r04, Approaches A-min and B achieved 85%–100% of offered writes, triggering 16–22 flushes. Approach A suffered 45%–78% write starvation (1,152 writes in equal_r01, 450 in equal_r04). Because Approach A's writer stalled on reader locks, its read path faced substantially lower background write and flush interference, explaining why raw read throughput appeared elevated in Approach A. Block E3 characterizes response to a *matched offered rate*, not equal realized work.

TABLE V
BLOCK E3 WRITE PROGRESS RECONCILIATION

Cell	Arm	Offered	Completed	Realized %	Flushes	Compactions
equal_r01	AMIN	2,350	2,350–2,351	100.0%	22	6
equal_r01	B	2,350	2,350–2,351	100.0%	22	6
equal_r01	A	2,350	1,152–1,293	49.0–55.0%	10–12	6
equal_r04	AMIN	2,100	1,801–2,101	85.8–100.0%	16–19	6
equal_r04	B	2,100	2,100–2,101	100.0%	19	6
equal_r04	A	2,100	450–454	21.4–21.6%	4	4
free_r01	AMIN	free	3,179–3,184	N/A	29–30	6
free_r01	B	free	3,164–3,257	N/A	29–30	6
free_r01	A	free	1,200–1,291	N/A	11–12	6

E. Block E4: Open-Loop Response Latencies

Table VI reconciles open-loop request accounting and response latency percentiles across arrival levels L_1 (4.86 ops/s), L_2 (9.72 ops/s), and L_3 (14.58 ops/s).

TABLE VI
BLOCK E4 OPEN-LOOP LATENCY AND REQUEST RECONCILIATION

Level	Arm	Sched	In-Win (N)	Late	Abnd	y_{p50} (ms)	y_{p95} (ms)
L_1	A	54–64	53–63	1	0	79.4–100.4	254.9–396.1
L_1	AMIN	54–64	53–63	1	0	70.9–97.5	206.6–391.3
L_1	B	54–64	53–63	1	0	62.4–101.7	210.6–387.9
L_2	A	101	99	2	0	60.8	388.4
L_2	AMIN	101	99	2	0	67.3	439.4
L_2	B	101	99	2	0	56.2	375.0
L_3	AMIN	145–151	142–151	0–3	0	71.3–111.5	528.3–852.6
L_3	B	145–151	142–151	0–3	0	71.3–107.9	450.8–748.5
L_3	A	145–151	127–132	14–16	3–4	**251.3–591.0**	**1,386–2,111**

At L_1 and L_2, all arms completed $> 98\%$ of requests in-window with aligned y_{p50} latencies (56–101 ms). At L_3, Approach A suffered severe tail inflation ($y_{p50} = 251$–591 ms, $y_{p95} = 1.38$–2.11 s), while Approaches A-min and B maintained $y_{p50} \leq 111.5$ ms. Total completed requests equal $N_{\text{in-window}} + N_{\text{late}}$, confirming that requests finishing during post-window drain account for the difference between C_{total} and N.

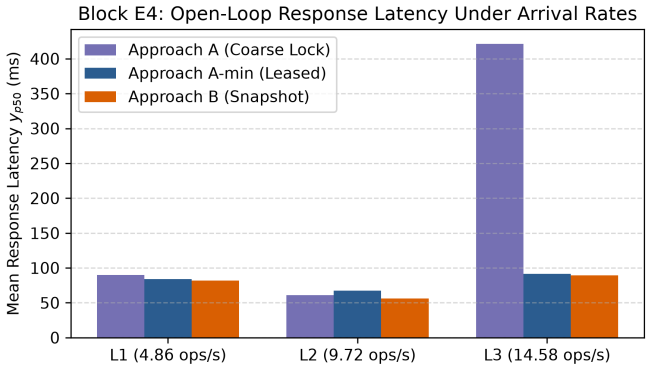


Fig. 3. Open-Loop Median Response Latency (y_{p50}) Across Arrival Rate Levels (Block E4, Windows 11 NTFS).

VII. Methodological Disclosures and Audits

A. Phase I Overlap Semantics Audit

Instrumentation in Phase I revealed positive `reads_during_publication` counts in Approach A despite holding an exclusive lock. Unit tests confirmed this was an artifact of reader drain delay: counters recorded reads that initiated under `RLock` before publication began and completed while the worker queued for `Lock`. Exactly zero reads executed while `Lock` was held.

B. Phase II E5 All-Miss Read Path Mechanism

In Block E5 ($h = 0$, all-miss), read throughput was unexpectedly high ($\sim 3,000$ ops/s) and p_{50} latency low (2.34–2.44 ms). Code investigation revealed that absent keys generated by `keystream.go` (index $\geq 10,001$) strictly exceeded the populated database range (0–9, 999). In `sstableReader.go`, binary search over the block index resolved immediately to the final block index. Readers inspected only the last 4KB data block, encountered EOF, and returned `ErrNotFound`. This final block remained pinned in OS page cache across queries, bypassing multi-block disk traversal. Block E5 does not validate an exhaustive multi-level scan.

C. Pebble Comparison Exclusion

Pebble benchmark runs in Block E7 were invalidated by two issues:

1) **Envelope Incompatibility**: `pebble_runner.exe` omitted `result.timestamps`, causing the orchestrator to flag all runs as `INVALID`.
2) **Buffer Exhaustion**: Fixed 1M-sample buffers per reader filled after 5 seconds at 1.88M ops/s, dropping over 10.45 million latency samples ($> 50\%$ of reads).

Pebble data are excluded from primary comparisons.

VIII. Threats to Validity

- **Truncated Factorial Execution**: Phase II halted at 60 of 330 planned trials under the stopping rule ($N_{\text{fail}} = 10$). While 18 complete matched triplets were preserved across 5 blocks, full factorial concurrency scaling ($r = 1 \ldots 32$ across all workloads) was not executed.
- **Unavailable Drift Baseline**: Due to runner omission of `"D3"` in `cmd/i7trial/main.go`, initial drift trials failed ($N_{\text{start}} = 0$). Campaign drift ratio cannot be computed, leaving environmental drift over the 26.4-minute run unbounded by the primary control.
- **Unrun Windows Race Detector**: Windows binaries were compiled without `-race` due to cgo toolchain unavailability. Thread safety was verified independently on Linux ext4.
- **Single-Host Storage Environment**: All Phase II trials operated on a single AMD Ryzen 7 laptop on NTFS with active OS page cache. Findings characterize in-process memory-lock bottlenecks rather than physical disk head serialization.
- **Unpooled Cross-Phase Datasets**: Phase I (WSL2 Linux) and Phase II (Windows NTFS) represent distinct experimental platforms. Their effect sizes are not statistically pooled.

IX. Conclusion

This study evaluated reader-writer synchronization architectures in Go LSM trees to determine whether simpler reader-lifetime management can capture the concurrency benefits of immutable reader snapshots. Our empirical findings demonstrate that shortening the manager-lock critical section using lightweight descriptor leasing (Approach A-min) eliminates Approach A's publication queue convoying and write starvation, achieving quantitative parity with immutable snapshot pinning (Approach B) within a $\pm 5\%$ equivalence band across 18 complete matched triplets. Furthermore, descriptor leasing avoids the 26%–37% throughput penalty observed in snapshot pinning under rapid reader-set churn. Storage engine architects can achieve the primary concurrency benefits of snapshot isolation through fine-grained descriptor leasing without incurring full snapshot lifecycle overhead.

References

[1] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.
[2] J. Dean and S. Ghemawat, "LevelDB: A fast and lightweight key/value database library by Google," GitHub repository, google/leveldb, 2011.
[3] Meta Platforms Inc., "RocksDB: A persistent key-value store," GitHub repository, facebook/rocksdb, 2013.
[4] W. Pugh, "Skip lists: A probabilistic alternative to balanced trees," *Communications of the ACM*, vol. 33, no. 6, pp. 668–676, 1990.
[5] P. A. Bernstein and N. Goodman, "Concurrency control in distributed database systems," *ACM Computing Surveys*, vol. 13, no. 2, pp. 185–221, 1981.
[6] L. Lu, T. S. Pillai, H. Gopalakrishnan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "WiscKey: Separating keys from values in SSD-conscious storage," *ACM Transactions on Storage*, vol. 13, no. 1, article 5, 2017.
[7] N. Dayan, M. Athanassoulis, and S. Idreos, "Monkey: Optimal navigable key-value stores," in *Proceedings of the 2017 ACM SIGMOD International Conference on Management of Data*, 2017, pp. 79–94.